Document number	P0411R0
Date	2015-07-07
Project	Programming Language C++, Library Evolution Working Group
Reply-to	Jonathan Wakely <cxx@kayari.org>

Separating Library Requirements and Preconditions

Introduction

The C++ Standard Library describes preconditions and requirements using the *Requires:* element (17.5.1.4 [structure.specifications]) and failure to meet these requirements usually results in undefined behaviour:

> **17.6.4.11 Requires paragraph [res.on.required]**
>
> Violation of the preconditions specified in a function's *Requires:* paragraph results in undefined behavior unless the function's *Throws:* paragraph specifies throwing an exception when the precondition is violated.

There are many requirements which can be statically checked and so could produce a diagnostic, rather than the entirely unpredictable outcome implied by undefined behaviour. I propose separate categories for requirements which produce a compile-time diagnostic when violated and for those which result in undefined behaviour when violated.

Motivation and Scope

Some library preconditions describe the types that a function template is expected to work with, for example the sequence container requirements say that `insert(p, t)` on a `vector` or `deque` requires `CopyAssignable`. Failure to meet that requirement is not a run-time violation of a contract, and will not give unpredictable or non-portable results, it will simply fail to compile. There is no good reason why that particular requirement should be be specified so that violating it is undefined. A future update to the Standard Library that uses Concepts would probably constrain such a function with a `requires` clause which would make violations ill-formed.

On the other hand, the requirement that the pointer passed to `operator delete` is not a null pointer and that its value was returned by `operator new` is a real precondition. In general it's impossible to check that requirement at compile-time, and violating it really can produce unpredictable results such as corrupting the heap, leading to crashes, memory leaks or data loss. Allowing that kind of precondition to result in undefined behaviour also allows tools such as valgrind or AddressSanitizer to perform extra checks to diagnose violations at run-time.

We already give useful guarantees about some types, e.g. attempting to copy a `unique_ptr` is guaranteed to be ill-formed, so users can be confident that attempts to misuse it will be caught by the compiler. But then we state other requirements in terms of undefined behaviour, which technically means users can't rely on the compiler telling them when they do something nonsensical. We should not place the burden of verifying such preconditions on users, with the threat of undefined behaviour if they fail to do so.

The aim of the proposal is not to require any implementations to change, but only to inform users whether violating requirements will result in a diagnostic, or whether it is their responsibility to check for precondition violations. For example, the following function template has undefined behaviour unless `is_copy_constructible_v<T>` is true:

```
template<typename T>
  std::optional<T> nonempty(const std::optional<T>& val) {
    return val ? val : std::optional<T>{ T{} };
  }
```

That means calling `nonempty(std::optional<std::unique_ptr<int>>{})` could theoretically compile, but crash at run-time, or do something worse. A paranoid author of that function would add a `static_assert` to verify that no undefined behaviour is possible, but that's totally unnecessary because in practice the `optional` copy constructor would be ill-formed and so the function would fail to compile anyway.

In keeping with the meaning of a *requires-expression* in the Concepts TS, I propose that the *Requires:* element be used for requirements on types that can be statically-enforced, and a new *Preconditions:* element be used for requirements that must be met to avoid undefined behaviour.

The Solution

The solution is conceptually simple, even if it involves a lot of changes to the specification of the library. Every *Requires:* that naturally produces a diagnostic anyway should stay as a *Requires:* element. This shouldn't require implementations to change, and simply standardizes existing practice. All other *Requires:* should be changed to *Preconditions:*, meaning that violations result in undefined behaviour.

In many cases it's obvious whether a *Requires:* element should be converted to *Preconditions:* but some cases are less obvious. For example, the generic `std::swap` requires `is_move_constructible` and `is_move_assignable` to be true, which are exactly the operations performed by the function. However the `std::swap` overload for arrays requires "`a[i]` shall be swappable with (17.6.3.2) `b[i]` for all `i` in the range `[0, N)`." The "swappable with" requirement includes postconditions on the values of the objects after they are swapped, which cannot be verified even at run-time for some types, let alone at compile-time.

In a few places we already say "Otherwise, the program is ill-formed" in *Requires* paragraphs, which becomes redundant with this proposal.

Several places in the library have the requirement "`Alloc` shall meet the requirements for an Allocator (17.6.3.5)." Although the allocator requirements are largely specified in terms of valid syntax, it's a very large interface to check for and there are also semantic requirement which cannot be verified statically. I propose that all such requirements should become *Preconditions*.

Some *Requires* paragraphs contain a mix of requirements and preconditions, so need to be split into two paragraphs.

A problem

Some *Requires* paragraphs currently state requirements in terms of concepts such as `CopyConstructible`, which has syntactic requirements but also semantic ones such as "the value of `v` is unchanged and is equivalent to `u`". The meaning of that phrase is not well-defined, and certainly can't be enforced at compile-time to produce a diagnostic.

It might make sense for some of those requirements to use a related trait such as `is_copy_constructible` instead. That trait only depends on properties that can be checked by the compiler, and so corresponds to the expressions that are actually evaluated and need to compile. However stating requirements only in terms of that trait would remove any semantic requirement on "same-ness" that might actually be important in some functions. It would be possible to add some weasel words so that only violations of syntactic parts of *Requires* paragraphs makes a program ill-formed, and violating semantic parts still results in undefined behaviour, but that might not actually improve on the status quo.

The current proposal does not attempt to resolve this problem, but it needs further thought.

Requirements stated in terms of `CopyConstructible` etc. have been left as *Requires* meaning they now require a diagnostic. Requirements in Clause 30 for `Lockable`, `TimedLockable` etc. have been changed to *Preconditions* because there are significant semantic aspects to those concepts.

Proposed Wording

Changes are relative to N4595.

Adjust 17.5.1.4 [structure.specifications]:

> -3- Descriptions of function semantics contain the following elements (as appropriate):
> — *Requires:* the requirements on types and valid expressions imposed by the function
> — ~~*Requires:*~~*Preconditions:* the preconditions for calling the function
> — *Effects:* the actions performed by the function
>
> [...]
>
> -4- Whenever the *Effects:* element specifies that the semantics of some function `F` are *Equivalent to* some code sequence, then the various elements are interpreted as follows. If `F`'s semantics specifies a *Requires:* element or a *Preconditions:* element, then ~~that requirement is~~ those requirements are logically imposed prior to the equivalent-to semantics. Next, the semantics of the code sequence are determined by the *Requires:*, *Preconditions:*, *Effects:*, *Synchronization:*, *Postconditions:*, *Returns:*, *Throws:*, *Complexity:*, *Remarks:*, *Error conditions:*, and *Notes:* specified for the function invocations contained in the code sequence. The value returned from `F` is specified by `F`'s *Returns:* element, or if `F` has no *Returns:* element, a non-`void` return from `F` is specified by the *Returns:* elements in the code sequence. If `F`'s semantics contains a *Throws:*, *Postconditions:*, or *Complexity:* element, then that supersedes any occurrences of that element in the code sequence.

Change Requires to Preconditions in the example in 17.6.3.2 [sappable.requirements]:

> *// ~~Requires:~~Preconditions:* `std::forward<T>(t)` *shall be swappable with* `std::forward<U>(u)`.

[...]

// ~~Requires:~~Preconditions: lvalues of `T` shall be swappable.

Adjust [res.on.required]:

17.6.4.11 Requires paragraph [res.on.required]

A program is ill-formed if it violates the requirements specified in a function's *Requires:* paragraph.

17.6.4.12 Preconditions paragraph [res.on.preconditions]

Violation of the preconditions specified in a function's ~~*Requires:*~~*Preconditions:* paragraph results in undefined behavior unless the function's *Throws:* paragraph specifies throwing an exception when the precondition is violated.

Change Requires to Preconditions in 18.6.2.1 [new.delete.single]:

```
void operator delete(void* ptr) noexcept;
void operator delete(void* ptr, std::size_t size) noexcept;
```

-10- *Effects:* [...]

-11- *Replaceable:* [...]

-12- ~~*Requires:*~~*Preconditions:* *ptr* shall be a null pointer or its value shall be a value returned by an earlier call to [...]

-13- ~~*Requires:*~~*Preconditions:* If an implementation has strict pointer safety (3.7.4.3) then `ptr` shall be a safely-derived pointer.

-14- ~~*Requires:*~~*Preconditions:* If present, the `std::size_t size` argument shall equal the size argument passed to the allocation function that returned `ptr`.

-15- *Required behavior:* [...]

-16- *Default behavior:* [...]

-17- *Default behavior:* [...]

-18- *Remarks:* [...]

```
void operator delete(void* ptr, const std::nothrow_t&) noexcept;
```

-19- *Effects:* [...]

-20- *Replaceable:* [...]

-21- ~~*Requires:*~~*Preconditions:* If an implementation has strict pointer safety (3.7.4.3) then `ptr` shall be a safely-derived pointer.

-22- *Default behavior:* [...]

Change Requires to Preconditions in 18.6.2.2 [new.delete.array]:

```
void operator delete[](void* ptr) noexcept;
void operator delete[](void* ptr, std::size_t size) noexcept;
```

-9- *Effects:* [...]

-10- *Replaceable:* [...]

-11- ~~*Requires:*~~*Preconditions:* *ptr* shall be a null pointer or its value shall be the value returned by an earlier call to [...]

-12- ~~*Requires:*~~*Preconditions:* If present, the `std::size_t size` argument shall equal the size argument passed to the allocation function that returned `ptr`.

-13- *Required behavior:* [...]

-14- ~~*Requires:*~~*Preconditions:* If an implementation has strict pointer safety (3.7.4.3) then `ptr` shall be a safely-derived pointer.

-15- *Default behavior:* [...]

```
void operator delete[](void* ptr, const std::nothrow_t&) noexcept;
```

-16- *Effects:* [...]

-17- *Replaceable:* [...]

-18- ~~*Requires:*~~*Preconditions:* If an implementation has strict pointer safety (3.7.4.3) then `ptr` shall be a safely-derived pointer.

-19- *Default behavior:* [...]

Change Requires to Preconditions in 18.6.2.3 [new.delete.placement]:

```
void operator delete(void* ptr, void*) noexcept;
```

-7- *Effects:* [...]

-8- ~~*Requires:*~~*Preconditions:* If an implementation has strict pointer safety (3.7.4.3) then `ptr` shall be a safely-derived pointer.

-9- *Remarks:* [...]

```
void operator delete[](void* ptr, void*) noexcept;
```

-10- *Effects:* [...]

-11- ~~*Requires:*~~*Preconditions:* If an implementation has strict pointer safety (3.7.4.3) then `ptr` shall be a safely-derived pointer.

-12- *Remarks:* [...]

Change Requires to Preconditions in 18.8.6 [propagation]

```
[[noreturn]] void rethrow_exception(exception_ptr p);
```

-9- ~~*Requires:*~~*Preconditions:* `p` shall not be a null pointer.

-10- *Throws:* the exception object to which `p` refers.

Change Requires to Preconditions in 20.2.2 [utility.swap]:

```
template <class T, size_t N>
  void swap(T (&a)[N], T (&b)[N]) noexcept(is_nothrow_swappable_v<T>);
```

-4- *Remarks:* This function shall not participate in overload resolution unless `is_swappable_v<T>` is `true`.

-3- ~~*Requires:*~~*Preconditions:* `a[i]` shall be swappable with (17.6.3.2) `b[i]` for all `i` in the range `[0, N)`.

-6- *Effects:* As if by `swap_ranges(a, a + N, b)`.

Change Requires to Preconditions for `pair::swap` in 20.4.2 [pairs.pair]:

```
void swap(pair& p) noexcept(see below );
```

-28- *Remarks:* The expression inside `noexcept` is equivalent to: [...]

-29- ~~*Requires:*~~*Preconditions:* `first` shall be swappable with (17.6.3.2) `p.first` and `second` shall be swappable with `p.second`.

-30- *Effects:* Swaps `first` with `p.first` and `second` with `p.second`.

Edit "ill-formed" sentences in 20.4.4 [pair.astuple]:

-?- *Requires:* `I < 2`.

-3- *Returns:* If `I == 0` returns a reference to `p.first`; if `I == 1` returns a reference to `p.second`~~; otherwise the program is ill-formed~~.

[...]

-4- *Requires:* `T` and `U` are distinct types. ~~Otherwise, the program is ill-formed.~~

[...]

-6- *Requires:* `T` and `U` are distinct types. ~~Otherwise, the program is ill-formed.~~

Change Requires to Preconditions for last set of constructors in 20.5.2.1 [tuple.cnstr]:

-27- ~~*Requires:*~~*Preconditions:* `Alloc` shall meet the requirements for an Allocator (17.6.3.5).

Change Requires to Preconditions in 20.5.2.3 [tuple.swap]:

-2- ~~*Requires:*~~*Preconditions:* Each element in `*this` shall be swappable with (17.6.3.2) the corresponding element in `rhs`.

Remove redundant sentence in 20.5.2.6 [tuple.helper]:

-2- *Requires:* `I < sizeof...(Types)`. ~~The program is ill-formed if `I` is out of bounds.~~

Remove redundant sentences in 20.5.2.7 [tuple.elem]:

-1- *Requires:* `I < sizeof...(Types)`. ~~The program is ill-formed if `I` is out of bounds.~~

[...]

-5- *Requires:* The type `T` occurs exactly once in `Types`. ~~Otherwise, the program is ill-formed.~~

Change Requires to Preconditions in 20.5.2.9 [tuple.traits]:

-?- ~~*Requires:*~~*Preconditions:* `Alloc` shall be an Allocator (17.6.3.5).

Move part of Requires to Preconditions in 20.6.3.4 [optional.object.swap]:

-1- *Requires:* ~~Lvalues of type `T` shall be swappable and~~ `is_move_constructible_v<T>` is `true`.

-?- *Preconditions:* Lvalues of type `T` shall be swappable.

Change Requires to Preconditions in 20.6.3.5 [optional.object.observe]:

-1- ~~*Requires:*~~*Preconditions:* `*this` contains a value.

[...]

-5- ~~*Requires:*~~*Preconditions:* `*this` contains a value.

[...]

-9- ~~*Requires:*~~*Preconditions:* `*this` contains a value.

Change Requires to Preconditions in 20.6.11 [optional.hash]:

-1- ~~*Requires:*~~*Preconditions:* The template specialization `hash<T>` shall meet the requirements of class template `hash` (20.13.14).

Drafting note: The `any` specification was changed by P0032R3 in Oulu, and the CopyConstructible requirements are now missing. The following edits will need to be updated.

Edit the Requires in 20.7.3.1 [any.cons]:

```
template<class ValueType>
any(ValueType&& value);
```

-6- Let `T` be equal to `decay_t<ValueType>`.

-7- *Requires:* ~~`T` shall satisfy the `CopyConstructible` requirements. If~~ `is_copy_constructible_v<T>` is `true` ~~false, the program is ill-formed~~.

-8- *Effects:* [...]

Edit the Requires in 20.7.3.2 [any.assign]:

```
template<class ValueType>
any& operator=(ValueType&& value);
```

-7- Let `T` be equal to `decay_t<ValueType>`.

-8- *Requires:* ~~`T` shall satisfy the `CopyConstructible` requirements. If~~ `is_copy_constructible_v<T>` is `true` ~~false, the program is ill-formed~~.

-9- *Effects:* [...]

Remove redundant sentence in 20.7.4 [any.nonmembers]:

```
template<class ValueType>
  ValueType any_cast(const any& operand);
template<class ValueType>
  ValueType any_cast(any& operand);
template<class ValueType>
  ValueType any_cast(any&& operand);
```

-2- *Requires:* `is_reference_v<ValueType>` is true or `is_copy_constructible_v<ValueType>` is true. ~~Otherwise the program is ill-formed.~~

-3- *Effects:* [...]

Change Requires to Preconditions in 20.8.2 [bitset.members]:

```
constexpr bool operator[](size_t pos) const;
```

-45- ~~*Requires:*~~*Preconditions:* `pos` shall be valid.

-46- *Returns:* `true` if the bit at position `pos` in `*this` has the value one, otherwise `false`.

-47- *Throws:* Nothing.

```
bitset<N>::reference operator[](size_t pos);
```

-48- ~~*Requires:*~~*Preconditions:* `pos` shall be valid.

-49- *Returns:* An object of type `bitset<N>::reference` such that `(*this)[pos] == this->test(pos)`, and such that `(*this)[pos] = val` is equivalent to `this->set(pos, val)`.

-50- *Throws:* Nothing.

Change Requires to Preconditions in 20.9.4 [util.dynamic.safety]:

```
void declare_reachable(void* p);
```

-2- ~~*Requires:*~~*Preconditions:* `p` shall be a safely-derived pointer (3.7.4.3) or a null pointer value.

-3- *Effects:* if `p` is not null, the complete object [...]

-4- *Throws:* May throw `bad_alloc` if [...]

```
template <class T> T* undeclare_reachable(T* p);
```

-5- ~~*Requires:*~~*Preconditions:* If `p` is not null, the complete object referenced by `p` shall have been previously declared reachable, and shall be live (3.8) from the time of the call until the last `undeclare_reachable(p)` call on the object.

-6- *Returns:* A safely-derived copy of `p` which shall compare equal to `p`.

-7- *Throws:* Nothing.

-8- [*Note:* It is expected that [...] *--end note*]

```
void declare_no_pointers(char* p, size_t n);
```

-9- ~~*Requires:*~~*Preconditions:* No bytes in the specified range are currently registered with `declare_no_pointers()`. [...]

-10- *Effects:* The `n` bytes starting at `p` no longer contain [...]

-11- *Throws:* Nothing.

-12- [*Note:* Under some conditions [...] *--end note*]

```
void declare_no_pointers(char* p, size_t n);
```

-13- ~~*Requires:*~~*Preconditions:* The same range must previously have been passed to `declare_no_pointers()`.

-14- *Effects:* Unregisters a range [...]

-15- *Throws:* Nothing.

Change Requires to Preconditions in 20.9.5 [ptr.align]:

```
void* align(size_t alignment, size_t size, void*& ptr, size_t& space);
```

-1- *Effects:* If it is possible [...]

-2- ~~*Requires:*~~*Preconditions:*

(2.1) --- `alignment` shall be a power of two

(2.2) --- `ptr` shall point to contiguous storage of at least `space` bytes.

-3- *Returns:* [...]

Change Requires to Preconditions in 20.9.9.1 [allocator.members]:

```
void deallocate(pointer p, size_type n);
```

-8- ~~*Requires:*~~*Preconditions:* `p` shall be a pointer value obtained from `allocate()`. `n` shall equal the value passed as the first argument to the invocation of allocate which returned `p`.

Change Requires to Preconditions in 20.9.11 [temporary.buffer]:

```
template <class T> void return_temporary_buffer(T* p);
```

-4- *Effects:* Deallocates the storage referenced by `p`.

-5- ~~*Requires:*~~*Preconditions:* `p` shall be a pointer value returned by an earlier call to `get_temporary_buffer` that has not been invalidated by an intervening call to `return_temporary_buffer(T*)`.

-6- *Throws:* Nothing.

Split Requires in 20.10.1.2.1 [unique.ptr.single.ctor]:

```
constexpr unique_ptr() noexcept;
```

-1- *Requires:* ~~`D` shall satisfy the requirements of `DefaultConstructible` (Table 19), and that construction shall not throw an exception.~~`is_pointer_v<D>` is `false` and `is_reference_v<D>` is `false` and `is_default_constructible_v<D>` is `true`.

-?- *Preconditions:* Initializing the stored deleter shall not throw an exception.

-2- *Effects:* Constructs a `unique_ptr` object that owns nothing, [...]

-3- *Postconditions:* `get == nullptr`. [...]

~~-4- *Remarks:* If this constructor is instantiated with a pointer or reference type for the template argument `D`, the program is ill-formed.~~

```
explicit unique_ptr(pointer p) noexcept;
```

-1- *Requires:* ~~`D` shall satisfy the requirements of `DefaultConstructible` (Table 19), and that construction shall not throw an exception.~~`is_pointer_v<D>` is `false` and `is_reference_v<D>` is `false` and `is_default_constructible_v<D>` is `true`.

-?- *Preconditions:* Initializing the stored deleter shall not throw an exception.

-2- *Effects:* Constructs a `unique_ptr` object which owns `p`, [...]

-3- *Postconditions:* `get == nullptr`. [...]

~~-4- *Remarks:* If this constructor is instantiated with a pointer or reference type for the template argument `D`, the program is ill-formed.~~

[...]

-12- *Requires:*

(12.1) -- If `D` is not a lvalue reference type then

(12.1.1) --- If `d` is an lvalue or const rvalue then the first constructor of this pair will be selected. ~~`D` shall satisfy the requirements of `CopyConstructible` (Table 21), and the copy constructor of `D` shall not throw an exception.~~ `is_copy_constructible_v<D>` shall be `true`. This `unique_ptr` will hold a copy of `d`.

(12.1.2) --- Otherwise, `d` is a non-const rvalue and the second constructor of this pair will be selected. ~~`D` shall satisfy the requirements of `MoveConstructible` (Table 20), and the move constructor of `D` shall not throw an exception.~~ `is_copy_constructible_v<D>` shall be `true`. This `unique_ptr` will hold a value move constructed from `d`.

(12.2) -- Otherwise `D` is an lvalue reference type. [...]

-?- *Preconditions:* If `D` is not an lvalue reference type then construction of the stored deleter shall not throw an exception.

-13- *Effects:* Constructs a `unique_ptr` object [...]

-14- *Postconditions:* `get() == p`. [...]

[*Example:*

[...]

--end example]

```
unique_ptr(unique_ptr&& u) noexcept;
```

-15- *Requires:* If `D` is not a reference type, `is_move_constructible_v<D>` is `true` ~~`D` shall satisfy the requirements of MoveConstructible (Table 20). Construction of the deleter from an rvalue of type D shall not throw an exception.~~

-?- *Preconditions:* Construction of the deleter from an rvalue of type D shall not throw an exception.

-16- *Effects* Constructs a `unique_ptr` by transferring ownership [...]

-17- *Postconditions:* `get()` yields the value `u.get()` yielded [...]

```
template <class U, class E> unique_ptr(unique_ptr<U, E>&& u) noexcept;
```

-18- *Requires:* If `E` is not a reference type, `is_constructible_v<D, E>` is `true` ~~construction of the deleter from an rvalue of type `E` shall be well formed and shall not throw an exception~~. Otherwise, `E` is a reference type and `is_constructible_v<D, E&>` is `true` ~~construction of the deleter from an lvalue of type `E` shall be well formed and shall not throw an exception~~.

-?- *Preconditions:* Construction of the deleter shall not throw an exception.

-19- *Remarks:* This constructor shall not participate in overload resolution unless: [...]

Split 20.10.1.2.2 [unique.ptr.single.dtor]:

```
~unique_ptr();
```

-1- *Requires:* The expression `get_deleter()(get())` shall be well formed~~, shall have well-defined behavior, and shall not throw exceptions~~. [*Note:* The use of `default_delete` requires `T` to be a complete type. *--end note*]

-?- *Preconditions:* The expression `get_deleter()(get())` shall have well-defined behavior, and shall not throw exceptions.

-2- *Effects:* If `get() == nullptr` there are no effects. Otherwise `get_deleter()(get())`.

Split 20.10.1.2.3 [unique.ptr.single.asgn]:

```
unique_ptr& operator=(unique_ptr&& u) noexcept;
```

-1- *Requires:* If `D` is not a reference type, `is_move_assignable_v<D>` is `true` ~~`D` shall satisfy the requirements of `MoveAssignable` (Table 22) and assignment of the deleter from an rvalue of type `D` shall not throw an exception~~. Otherwise, `D` is a reference type; `is_copy_assignable_v<remove_reference_t<D>>` is `true`. ~~`remove_reference_t<D>` shall satisfy the `CopyAssignable` requirements and assignment of the deleter from an lvalue of type `D` shall not throw an exception.~~

-?- *Preconditions:* If `D` is not a reference type, assignment of the deleter from an rvalue of type `D` shall not throw an exception. Otherwise, `D` is a reference type; assignment of the deleter from an lvalue of type `D` shall not throw an exception.

-2- *Effects:* Transfers ownership from `u` to `*this` as if by calling `reset(u.release())` followed by `get_deleter() = std::forward<D>(u.get_deleter())`.

-3- *Returns:* `*this`.

```
template <class U, class E> unique_ptr& operator=(unique_ptr<U, E>&& u) noexcept;
```

-4- ~~*Requires:*~~*Preconditions:* If `E` is not a reference type, assignment of the deleter from an rvalue of type `E` ~~shall be well-formed and~~ shall not throw an exception. Otherwise, `E` is a reference type and assignment of the deleter from an lvalue of type `E` ~~shall be well-formed and~~ shall not throw an exception.

-5- *Remarks:* This operator shall not participate in overload resolution unless:

(5.1) -- `unique_ptr<U, E>::pointer` is implicitly convertible to `pointer`, and

(5.2) -- `U` is not an array type, and

(5.3) -- `is_assignable<D&, E&&>::value` is `true`.

Change Requires to Preconditions in 20.10.1.2.4 [unique.ptr.single.observers]:

```
add_lvalue_reference_t<T> operator*() const;
```

-1- ~~*Requires:*~~*Preconditions:* `get() != nullptr`.

-2- *Returns:* `*get()`.

```
pointer operator->() const noexcept;
```

-3- ~~*Requires:*~~*Preconditions:* `get() != nullptr`.

-4- *Returns:* `get()`.

-5- *Note:* use typically requires that `T` be a complete type.

Change Requires to Preconditions in 20.10.1.2.5 [unique.ptr.single.modifiers]:

```
void reset(pointer p = pointer()) noexcept;
```

-3- ~~*Requires:*~~*Preconditions:* The expression `get_deleter()(get())` ~~shall be well formed,~~ shall have well-defined behavior, and shall not throw exceptions.

-4- *Effects:* Assigns `p` to the stored pointer, and then [...]

-5- *Postcondition:* `get() == p`. [...]

```
void swap(unique_ptr& u) noexcept;
```

-6- ~~*Requires:*~~*Preconditions:* `get_deleter()` shall be swappable (17.6.3.2) and shall not throw an exception under `swap`.

-7- *Effects:* Invokes `swap` on the stored pointers and on the stored deleters of `*this` and `u`.

Change Requires to Preconditions in 20.10.1.3.3 [unique.ptr.runtime.observers]:

```
T& operator[](size_t i) const;
```

-1- ~~*Requires:*~~*Preconditions:* `i <` the number of elements in the array to which the stored pointer points.

-2- *Returns:* `get()[i]`.

Change Requires to Preconditions in 20.10.1.5 [unique.ptr.special]:

```
template <class T1, class D1, class T2, class D2>
  bool operator<(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

-5- ~~*Requires:*~~*Preconditions:* Let `CT` denote `common_type_t<typename unique_ptr<T1, D1>::pointer, typename unique_ptr<T2, D2>::pointer>`. Then the specialization `less<CT>` shall be a function object type (20.12) that induces a strict weak ordering (25.5) on the pointer values.

-6- *Returns:* `less<CT>()(x.get(), y.get())`.

[...]

```
template <class T, class D>
  bool operator<(const unique_ptr<T, D>& x, nullptr_t);
template <class T, class D>
  bool operator<(nullptr_t, const unique_ptr<T, D>& x);
```

-13- ~~*Requires:*~~*Preconditions:* The specialization `less<unique_ptr<T, D>::pointer>` shall be a function object type (20.12)

that induces a strict weak ordering (25.5) on the pointer values.

-14- *Returns:* The first function template returns [...]

Split Requires in 20.10.2.2.1 [util.smartptr.shared.const]:

```
template<class Y> explicit shared_ptr(Y* p);
```

-4- *Requires:* `p` shall be convertible to `T*`. `Y` shall be a complete type. The expression `delete p` shall be well formed~~, shall have well defined behavior, and shall not throw exceptions~~.

-?- *Preconditions:* The expression `delete p` shall have well defined behavior, and shall not throw exceptions.

-5- *Effects:* Constructs a `shared_ptr` object that owns the pointer `p`. Enables `shared_from_this` with `p`. If an exception is thrown, `delete p` is called.

[...]

```
template<class Y, class D> shared_ptr(Y* p, D d);
template<class Y, class D, class A> shared_ptr(Y* p, D d, A a);
template <class D> shared_ptr(nullptr_t p, D d);
template <class D, class A> shared_ptr(nullptr_t p, D d, A a);
```

-?- *Requires:* `p` shall be convertible to `T*`. `is_copy_constructible_v<D>` shall be `true`. The expression `d(p)` shall be well formed.

-8- ~~*Requires:*~~*Preconditions:* ~~`p` shall be convertible to `T*`. `D` shall be `CopyConstructible`.~~ The copy constructor and destructor of `D` shall not throw exceptions. The expression `d(p)` ~~shall be well formed,~~ shall have well defined behavior, and shall not throw exceptions. `A` shall be an allocator (17.6.3.5). The copy constructor and destructor of `A` shall not throw exceptions.

-9- *Effects:* Constructs a `shared_ptr` object that owns the object `p` and the deleter `d`. The first and second constructors enable `shared_from_this` with `p`. The second and fourth constructors shall use a copy of `a` to allocate memory for internal use. If an exception is thrown, `d(p)` is called.

Change Requires to Preconditions in 20.10.2.2.5 [util.smartptr.shared.obs]:

```
T& operator*() const noexcept;
```

-2- ~~*Requires:*~~*Preconditions:* `get() != 0`.

-3- *Returns:* `*get()`.

-4- *Remarks:* When `T` is (possibly cv-qualified) `void`, it is unspecified whether this member function is declared. If it is declared, it is unspecified what its return type is, except that the declaration (although not necessarily the definition) of the function shall be well formed.

```
T* operator->() const noexcept;
```

-5- ~~*Requires:*~~*Preconditions:* `get() != 0`.

-6- *Returns:* `get()`.

Split Requires in 20.10.2.2.6 [util.smartptr.shared.create]:

```
template<class T, class... Args> shared_ptr<T> make_shared(Args&&... args);
template<class T, class A, class... Args>
  shared_ptr<T> allocate_shared(const A& a, Args&&... args);
```

-1- *Requires:* The expression `::new (pv) T(std::forward<Args>(args)...)`, where `pv` has type `void*` and points to storage suitable to hold an object of type `T`, shall be well formed. ~~`A` shall be an allocator (17.6.3.5). The copy constructor and destructor of `A` shall not throw exceptions.~~

-?- *Preconditions:* `A` shall be an allocator (17.6.3.5). The copy constructor and destructor of `A` shall not throw exceptions.

-2- *Effects:* Allocates memory [...]

Split Requires in 20.10.2.2.9 [util.smartptr.shared.cast]:

```
template<class T, class U> shared_ptr<T> dynamic_pointer_cast(const shared_ptr<U>& r) noexcept;
```

-5- *Requires:* The expression `dynamic_cast<T*>(r.get())` shall be well formed ~~and shall have well defined behavior~~.

-?- *Preconditions:* The expression `dynamic_cast<T*>(r.get())` shall have well defined behavior.

Change Requires to Preconditions in 20.10.2.6 [util.smartptr.shared.atomic]:

```
template<class T>
bool atomic_is_lock_free(const shared_ptr<T>* p);
```

-3- ~~*Requires:*~~*Preconditions:* `p` shall not be null.

-4- *Returns:* `true` if atomic access to `*p` is lock-free, `false` otherwise.

-5- *Throws:* Nothing.

```
template<class T>
shared_ptr<T> atomic_load(const shared_ptr<T>* p);
```

-6- ~~*Requires:*~~*Preconditions:* `p` shall not be null.

-7- *Returns:* `atomic_load_explicit(p, memory_order_seq_cst)`.

-8- *Throws:* Nothing.

```
template<class T>
shared_ptr<T> atomic_load_explicit(const shared_ptr<T>* p, memory_order mo);
```

-9- ~~*Requires:*~~*Preconditions:* `p` shall not be null.

-10- ~~*Requires:*~~*Preconditions:* `mo` shall not be `memory_order_release` or `memory_order_acq_rel`.

-11- *Returns:* `*p`.

-12- *Throws:* Nothing.

```
template<class T>
void atomic_store(shared_ptr<T>* p, shared_ptr<T> r);
```

-13- ~~*Requires:*~~*Preconditions:* `p` shall not be null.

-14- *Effects:* As if by `atomic_store_explicit(p, r, memory_order_seq_cst)`.

-15- *Throws:* Nothing.

```
template<class T>
void atomic_store_explicit(shared_ptr<T>* p, shared_ptr<T> r, memory_order mo);
```

-16- ~~*Requires:*~~*Preconditions:* `p` shall not be null.

-17- ~~*Requires:*~~*Preconditions:* `mo` shall not be `memory_order_acquire` or `memory_order_acq_rel`.

-18- *Effects:* As if by `p->swap(r)`.

-19- *Throws:* Nothing.

```
template<class T>
shared_ptr<T> atomic_exchange(shared_ptr<T>* p, shared_ptr<T> r);
```

-20- ~~*Requires:*~~*Preconditions:* `p` shall not be null.

-21- *Returns:* `atomic_exchange_explicit(p, r, memory_order_seq_cst)`.

-22- *Throws:* Nothing.

```
template<class T>
  shared_ptr<T> atomic_exchange_explicit(shared_ptr<T>* p, shared_ptr<T> r,
                                         memory_order mo);
```

-23- ~~*Requires:*~~*Preconditions:* `p` shall not be null.

-24- *Effects:* As if by `p->swap(r)`.

-25- *Returns:* The previous value of `*p`.

-26- *Throws:* Nothing.

```
template<class T>
```

```
  bool atomic_compare_exchange_weak(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
```

-27- ~~*Requires:*~~*Preconditions:* `p` shall not be null and `v` shall not be null.

-28- *Returns:* `atomic_compare_exchange_weak_explicit(p, v, w, memory_order_seq_cst, memory_order_seq_cst)`.

-29- *Throws:* Nothing.

```
template<class T>
  bool atomic_compare_exchange_strong(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
```

-30- *Returns:* `atomic_compare_exchange_strong_explicit(p, v, w, memory_order_seq_cst, memory_order_seq_cst)`.

```
template<class T>
  bool atomic_compare_exchange_weak_explicit(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
    memory_order success, memory_order failure);
template<class T>
  bool atomic_compare_exchange_strong_explicit(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
    memory_order success, memory_order failure);
```

-31- ~~*Requires:*~~*Preconditions:* `p` shall not be null and `v` shall not be null.

-32- ~~*Requires:*~~*Preconditions:* `failure` shall not be `memory_order_release`, `memory_order_acq_rel`, or stronger than `success`.

-33- *Effects:* If `*p` is equivalent to `*v`, assigns `w` to `*p` and has synchronization semantics corresponding to the value of `success`, otherwise assigns `*p` to `*v` and has synchronization semantics corresponding to the value of `failure`.

Change Requires to Preconditions in 20.10.2.7 [util.smartptr.hash]:

-2- ~~*Requires:*~~*Preconditions:* The specialization `hash<typename UP::pointer>` shall be well-formed and well-defined, and shall meet the requirements of class template `hash` (20.12.14).

Change Requires to Preconditions in 20.11.2.2 [memory.resource.prot]:

```
virtual void* do_allocate(size_t bytes, size_t alignment) = 0;
```

-1- ~~*Requires:*~~*Preconditions:* Alignment shall be a power of two.

[...]

```
virtual void do_deallocate(void* p, size_t bytes, size_t alignment) = 0;
```

-4- ~~*Requires:*~~*Preconditions:* `p` shall have been returned from a prior call to `allocate(bytes, alignment)` on a memory resource equal to `*this`, and the storage at `p` shall not yet have been deallocated.

Change Requires to Preconditions in 20.11.3.1 [memory.polymorphic.allocator.ctor]:

```
polymorphic_allocator(memory_resource* r);
```

-2- ~~*Requires:*~~*Preconditions:* `r` is non-null.

-3- *Effects:* Sets `memory_rsrc` to `r`.

Change Requires to Preconditions in 20.11.3.2 [memory.polymorphic.allocator.mem]:

```
void deallocate(Tp* p, size_t n);
```

-2- ~~*Requires:*~~*Preconditions:* `p` was allocated from a memory resource `x`, equal to `*memory_rsrc`, using `x.allocate(n * sizeof(Tp), alignof(Tp))`.

-3- *Effects:* Equivalent to `memory_rsrc->deallocate(p, n * sizeof(Tp), alignof(Tp))`.

Change Requires to Preconditions in 20.11.5.3 [memory.resource.pool.ctor]:

```
synchronized_pool_resource(const pool_options& opts, memory_resource* upstream);
unsynchronized_pool_resource(const pool_options& opts, memory_resource* upstream);
```

-1- ~~*Requires:*~~*Preconditions:* `upstream` is the address of a valid memory resource.

-2- *Effects:* [...]

Change Requires to Preconditions in 20.11.6.1 [memory.resource.monotonic.buffer.ctor]:

```
explicit monotonic_buffer_resource(memory_resource* upstream);
monotonic_buffer_resource(size_t initial_size, memory_resource* upstream);
```

-1- ~~*Requires:*~~*Preconditions:* `upstream` shall be the address of a valid memory resource. `initial_size`, if specified, shall be greater than zero.

-2- *Effects:* Sets `upstream_rsrc` to `upstream` and `current_buffer` to `nullptr`. If `initial_size` is specified, sets `next_buffer_size` to at least `initial_size`; otherwise sets `next_buffer_size` to an implementation-defined size.

```
monotonic_buffer_resource(void* buffer, size_t buffer_size, memory_resource* upstream);
```

-3- ~~*Requires:*~~*Preconditions:* `upstream` shall be the address of a valid memory resource. `buffer_size` shall be no larger than the number of bytes in `buffer`.

-4- *Effects*: Sets `upstream_rsrc` to `upstream` and [...]

Drafting note: The specification for `not_fn` was changed in Oulu, this wording will need updating.

Split Requires in 20.12.9 [func.not_fn]:

-2- *Requires:* `is_constructible<FD, F>::value` shall be `true`. ~~`fd` shall be a callable object (20.12.1).~~

-?- *Preconditions:* `fd` shall be a callable object (20.12.1).

Split Requires in 20.12.10.3 [func.bind.bind]:

```
template<classF, class... BoundArgs>
  unspecified bind(F&& f, BoundArgs&&... bound_args);
```

-2- *Requires:* `is_constructible<FD, F>::value` shall be `true`. For each `Ti` in `BoundArgs`, `is_constructible<TiD, Ti>::value` shall be `true`. ~~`INVOKE(fd, w1, w2, ..., wN)` (20.12.2) shall be a valid expression for some values *w1, w2, ..., wN,* where `N == sizeof...(bound_args)`. The cv-qualifiers *cv* of the call wrapper `g`, as specified below, shall be neither `volatile` nor `const volatile`.~~

-?- *Preconditions:* `INVOKE(fd, w1, w2, ..., wN)` (20.12.2) shall be a valid expression for some values *w1, w2, ..., wN,* where `N == sizeof...(bound_args)`. The cv-qualifiers *cv* of the call wrapper `g`, as specified below, shall be neither `volatile` nor `const volatile`.

[...]

```
template<class R, class F, class... BoundArgs>
  unspecified bind(F&& f, BoundArgs&&... bound_args);
```

-6- *Requires:* `is_constructible<FD, F>::value` shall be `true`. For each `Ti` in `BoundArgs`, `is_constructible<TiD, Ti>::value` shall be `true`. ~~`INVOKE(fd, w1, w2, ..., wN)` (20.12.2) shall be a valid expression for some values *w1, w2, ..., wN,* where `N == sizeof...(bound_args)`. The cv-qualifiers *cv* of the call wrapper `g`, as specified below, shall be neither `volatile` nor `const volatile`.~~

-?- *Preconditions:* `INVOKE(fd, w1, w2, ..., wN)` (20.12.2) shall be a valid expression for some values *w1, w2, ..., wN,* where `N == sizeof...(bound_args)`. The cv-qualifiers *cv* of the call wrapper `g`, as specified below, shall be neither `volatile` nor `const volatile`.

Change one Requires to Preconditions in 20.12.13.2 [func.searchers.boyer_moore]:

-1- *Requires:* The value type of `RandomAccessIterator1` shall meet the `DefaultConstructible` requirements, the `CopyConstructible` requirements, and the `CopyAssignable` requirements.

-2- ~~*Requires:*~~*Preconditions:* For any two values `A` and `B` of the type `iterator_traits<RandomAccessIterator1>::value_type`, if `pred(A,B)==true`, then `hf(A)==hf(B)` shall be `true`.

-3- *Effects:* Constructs a [...]

Change one Requires to Preconditions in 20.12.13.3 [func.searchers.boyer_moore_horspool]:

-1- *Requires:* The value type of `RandomAccessIterator1` shall meet the `DefaultConstructible` requirements, the `CopyConstructible` requirements, and the `CopyAssignable` requirements.

-2- ~~*Requires:*~~*Preconditions:* For any two values `A` and `B` of the type `iterator_traits<RandomAccessIterator1>::value_type`, if `pred(A,B)==true`, then `hf(A)==hf(B)` shall be `true`.

-3- *Effects:* Constructs a [...]

Should the following Remarks for `duration` be changed to Requires?

Change one Requires to Preconditions in 20.15.5 [time.duration]:

-2- *Requires:* `Rep` shall be an arithmetic type or a class emulating an arithmetic type.

-3- *Remarks:* If `duration` is instantiated with a `duration` type for the template argument `Rep`, the program is ill-formed.

-4- *Remarks:* If `Period` is not a specialization of `ratio`, the program is ill-formed.

-5- *Remarks:* If `Period::num` is not positive, the program is ill-formed.

-6- ~~*Requires:*~~*Preconditions:* Members of `duration` shall not throw exceptions other than those thrown by the indicated operations on their representations.

The following requirement on `system_clock::rep` isn't a requirement on users but on implementations, so should not use Requires or Preconditions.

Change Requires to Remarks in 20.15.7.1 [time.clock.system]:

```
typedef unspecified system_clock::rep;
```

-1- ~~*Requires:*~~*Remarks:* `system_clock::duration::min() < system_clock::duration::zero()` shall be `true`. [*Note:* This implies that `rep` is a signed type. –*end note*]

It's not clear to me whether Preconditions is right for the next items. Are these requirements on program-defined specializations of `char_traits` or on the predefined ones? If the former, we don't want implementations to have to diagnose invalid program-defined specializations. If the latter, maybe all three *Requires* below should use *Remarks*.

Change Requires to Preconditions in 21.2.2 [char.traits.typedefs]:

```
typedef INT_T int_type;
```

-2- ~~*Requires:*~~*Preconditions:* For a certain character container type `char_type`, a related container type `INT_T` shall be a type or class which can represent all of the valid characters converted from the corresponding `char_type` values, as well as an end-of-file value, `eof()`. The type `int_type` represents a character container type which can hold end-of-file to be used as a return type of the iostream class member functions.

```
typedef implementation-defined off_type;
typedef implementation-defined pos_type;
```

-3- ~~*Requires:*~~*Preconditions:* Requirements for `off_type` and `pos_type` are described in 27.2.2 and 27.3.

```
typedef STATE_T state_type;
```

-4- Requires: `state_type` shall meet the requirements of `CopyAssignable` (Table 23), `CopyConstructible` (Table 21), and `DefaultConstructible` (Table 19) types.

Change all Requires to Preconditions in 21.3.1.2 [string.cons]:

```
basic_string(const charT* s, size_type n,
             const Allocator& a = Allocator());
```

-7- ~~*Requires:*~~*Preconditions:* `s` points to an array of at least `n` elements of `charT`.

-8- *Effects:* Constructs an object [...]

```
basic_string(const charT* s, const Allocator& a = Allocator());
```

-9- ~~*Requires:*~~*Preconditions:* `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

-10- *Effects:* Constructs an object [...]

-11- *Remarks:* Uses `traits::length()`.

```
basic_string(size_type n, charT c, const Allocator& a = Allocator());
```

-12- ~~*Requires:*~~*Preconditions:* `n < npos`.

-13- *Effects:* Constructs an object [...]

Change all Requires to Preconditions in 21.3.1.5 [string.access]:

```
const_reference operator[](size_type pos) const;
reference       operator[](size_type pos);
```

-1- ~~*Requires:*~~*Preconditions:* `pos <= size()`.

-2- *Returns:* `*(begin() + pos)` if `pos < size()`. Otherwise, returns a reference to an object of type `charT` with value `charT()`, where modifying the object leads to undefined behavior.

-3- *Throws:* Nothing.

-4- *Complexity:* Constant time.

```
const_reference at(size_type pos) const;
reference       at(size_type pos);
```

-5- *Throws:* `out_of_range` if `pos >= size()`.

-6- *Returns:* `operator[](pos)`.

```
const charT& front() const;
charT& front();
```

-7- ~~*Requires:*~~*Preconditions:* `!empty()`.

-8- *Effects:* Equivalent to `operator[](0)`.

```
const charT& back() const;
charT& back();
```

-9- ~~*Requires:*~~*Preconditions:* `!empty()`.

-10- *Effects:* Equivalent to `operator[](size() - 1)`.

Change all Requires to Preconditions in 21.3.1.6.2 [string::append]:

```
basic_string&
  append(const charT* s, size_type n);
```

-6- ~~*Requires:*~~*Preconditions:* `s` points to an array of at least `n` elements of `charT`.

-7- *Throws:* `length_error` if `size() + n > max_size()`.

-8- *Effects:* The function replaces the string controlled by `*this` with a string of length `size() + n` whose first `size()` elements are a copy of the original string controlled by `*this` and whose remaining elements are a copy of the initial `n` elements of `s`.

-9- *Returns:* `*this`.

```
basic_string& append(const charT* s);
```

-10- ~~*Requires:*~~*Preconditions:* `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

-11- *Effects:* Calls `append(s, traits::length(s))`.

-12- *Returns:* *this.

```
basic_string& append(size_type n, charT c);
```

-13- *Effects:* Equivalent to `append(basic_string(n, c))`.

-14- *Returns:* *this.

```
template<class InputIterator>
  basic_string& append(InputIterator first, InputIterator last);
```

-15- ~~*Requires:*~~*Preconditions:* `[first, last)` is a valid range.

-16- *Effects:* Equivalent to `append(basic_string(first, last))`.

-17- *Returns:* `*this`.

Change all Requires to Preconditions in 21.3.1.6.3 [string::assign]:

```
basic_string& assign(const charT* s, size_type n);
```

-8- ~~*Requires:*~~*Preconditions:* `s` points to an array of at least `n` elements of `charT`.

-9- *Throws:* `length_error` if `n > max_size()`.

-10- *Effects:* Replaces the string controlled by `*this` with a string of length `n` whose elements are a copy of those pointed to by `s`.

-11- *Returns:* `*this`.

```
basic_string& assign(const charT* s);
```

-12- ~~*Requires:*~~*Preconditions:* `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

-13- *Effects:* Calls `assign(s, traits::length(s))`.

-14- *Returns:* `*this`.

Change all Requires to Preconditions in 21.3.1.6.4 [string::insert]:

```
basic_string&
insert(size_type pos, const charT* s, size_type n);
```

-5- ~~*Requires:*~~*Preconditions:* `s` points to an array of at least `n` elements of `charT`.

-6- *Throws:* `out_of_range` if `pos > size()` or `length_error` if `size() + n > max_size()`.

-7- *Effects:* Replaces the string controlled by `*this` with a string of length `size() + n` whose first `pos` elements are a copy of the initial elements of the original string controlled by `*this` and whose next `n` elements are a copy of the elements in `s` and whose remaining elements are a copy of the remaining elements of the original string controlled by `*this`.

-8- *Returns:* `*this`.

```
basic_string&
insert(size_type pos, const charT* s);
```

-9- ~~*Requires:*~~*Preconditions:* `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

-10- *Effects:* Equivalent to: `return insert(pos, s, traits::length(s));`

```
basic_string&
insert(size_type pos, size_type n, charT c);
```

-11- *Effects:* Equivalent to `insert(pos, basic_string(n, c))`.

-12- *Returns:* `*this`.

```
iterator insert(const_iterator p, charT c);
```

-13- ~~*Requires:*~~*Preconditions:* `p` is a valid iterator on `*this`.

-14- *Effects:* Inserts a copy of `c` before the character referred to by `p`.

-15- *Returns:* An iterator which refers to the copy of the inserted character.

```
iterator insert(const_iterator p, size_type n, charT c);
```

-16- ~~*Requires:*~~*Preconditions:* `p` is a valid iterator on `*this`.

-17- *Effects:* Inserts `n` copies of `c` before the character referred to by `p`.

-18- *Returns:* An iterator which refers to the copy of the first inserted character, or `p` if `n == 0`.

```
template<class InputIterator>
iterator insert(const_iterator p, InputIterator first, InputIterator last);
```

-19- ~~*Requires:*~~*Preconditions:* `p` is a valid iterator on `*this`. `[first,last)` is a valid range.

-20- *Effects:* Equivalent to `insert(p - begin(), basic_string(first, last))`.

-21- *Returns:* An iterator which refers to the copy of the first inserted character, or `p` if `first == last`.

Change all Requires to Preconditions in 21.3.1.6.5 [string::erase]:

```
iterator erase(const_iterator first, const_iterator last);
```

-8- ~~*Requires:*~~*Preconditions:* `first` and `last` are valid iterators on `*this`, defining a range `[first,last)`.

-9- *Throws:* Nothing.

-10- *Effects:* Removes the characters in the range `[first,last)`.

-11- *Returns:* An iterator which points to the element pointed to by last prior to the other elements being erased. If no such element exists, `end()` is returned.

```
void pop_back();
```

-12- ~~*Requires:*~~*Preconditions:* `!empty()`.

-13- *Throws:* Nothing.

-14- *Effects:* Equivalent to `erase(size() - 1, 1)`.

Note: Send help, I'm trapped in a WG21 proposal factory.

Change all Requires to Preconditions in 21.3.1.6.6 [string::replace]:

-5- ~~*Requires:*~~*Preconditions:* `s` points to an array of at least `n2` elements of `charT`.

[...]

-9- ~~*Requires:*~~*Preconditions:* `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

[...]

-13- ~~*Requires:*~~*Preconditions:* `[begin(), i1)` and `[i1, i2)` are valid ranges.

[...]

-16- ~~*Requires:*~~*Preconditions:* `[begin(), i1)` and `[i1, i2)` are valid ranges and `s` points to an array of at least `n` elements of `charT`.

[...]

-19- ~~*Requires:*~~*Preconditions:* `[begin(), i1)` and `[i1, i2)` are valid ranges and `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

[...]

-22- ~~*Requires:*~~*Preconditions:* `[begin(), i1)` and `[i1, i2)` are valid ranges.

[...]

-25- ~~*Requires:*~~*Preconditions:* `[begin(), i1)`, `[i1, i2)` and `[j1, j2)` are valid ranges.

[...]

-25- ~~*Requires:*~~*Preconditions:* `[begin(), i1)` and `[i1, i2)` are valid ranges.

Change all Requires to Preconditions in 21.3.1.7.1 [string.accessors]:

```
const charT* c_str() const noexcept;
const charT* data() const noexcept;
```

-1- *Returns:* A pointer `p` such that `p + i == &operator[](i)` for each `i` in `[0, size()]`.

-2- *Complexity:* Constant time.

-3- ~~*Requires:*~~*Preconditions:* The program shall not alter any of the values stored in the character array.

```
charT* data() noexcept;
```

-4- *Returns:* A pointer `p` such that `p + i == &operator[](i)` for each `i` in `[0, size()]`.

-5- *Complexity:* Constant time.

-6- ~~*Requires:*~~*Preconditions:* The program shall not alter the value stored at `p + size()`.

Change Requires to Preconditions in 21.3.1.7.2 [string::find]:

```
size_type find(const charT* s, size_type pos = 0) const;
```

-5- ~~*Requires:*~~*Preconditions:* `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

-6- *Returns:* `find(basic_string(s), pos)`.

Change Requires to Preconditions in 21.3.1.7.3 [string::rfind]:

```
size_type rfind(const charT* s, size_type pos = npos) const;
```

-5- ~~*Requires:*~~*Preconditions:* `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

-6- *Returns:* `rfind(basic_string(s), pos)`.

Change Requires to Preconditions in 21.3.1.7.4 [string::find.first.of]:

```
size_type find_first_of(const charT* s, size_type pos = 0) const;
```

-5- ~~*Requires:*~~*Preconditions:* `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

-6- *Returns:* `find_first_of(basic_string(s), pos)`.

Change Requires to Preconditions in 21.3.1.7.5 [string::find.last.of]:

```
size_type find_last_of(const charT* s, size_type pos = npos) const;
```

-5- ~~*Requires:*~~*Preconditions:* `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

-6- *Returns:* `find_last_of(basic_string(s), pos)`.

Change Requires to Preconditions in 21.3.1.7.6 [string::find.first.not.of]:

```
size_type find_first_not_of(const charT* s, size_type pos = 0) const;
```

-5- ~~*Requires:*~~*Preconditions:* `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

-6- *Returns:* `find_first_not_of(basic_string(s), pos)`.

Change Requires to Preconditions in 21.3.1.7.7 [string::find.last.not.of]:

```
size_type find_last_not_of(const charT* s, size_type pos = npos) const;
```

-5- ~~*Requires:*~~*Preconditions:* `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

-6- *Returns:* `find_last_not_of(basic_string(s), pos)`.

Change Requires to Preconditions in 21.3.2.2 [string::operator==]:

```
template<class charT, class traits, class Allocator>
  bool operator==(const basic_string<charT,traits,Allocator>& lhs,
                  const charT* rhs);
```

-3- ~~*Requires:*~~*Preconditions:* `rhs` points to an array of at least `traits::length(rhs) + 1` elements of `charT`.

-4- *Returns:* `lhs.compare(rhs) == 0`.

Change Requires to Preconditions in 21.3.2.3 [string::operator!=]:

```
template<class charT, class traits, class Allocator>
  bool operator!=(const basic_string<charT,traits,Allocator>& lhs,
                  const charT* rhs);
```

-3- ~~*Requires:*~~*Preconditions:* `rhs` points to an array of at least `traits::length(rhs) + 1` elements of `charT`.

-4- *Returns:* `lhs.compare(rhs) != 0`.

Change Requires to Preconditions in 21.4.2.1 [string.view.cons]:

```
constexpr basic_string_view(const charT* str);
```

-4- ~~*Requires:*~~*Preconditions:* `[str, str + traits::length(str))` is a valid range.

```
constexpr basic_string_view(const charT* str, size_type len);
```

-7- ~~*Requires:*~~*Preconditions:* `[str, str + len)` is a valid range.

Change all Requires to Preconditions in 21.4.2.4 [string.view.capacity]:

```
constexpr const_reference operator[](size_type pos) const;
```

-1- ~~*Requires:*~~*Preconditions:* `pos <= size()`.

[...]

```
constexpr const_reference front() const;
```

-7- ~~*Requires:*~~*Preconditions:* `!empty()`.

[...]

```
constexpr const_reference back() const;
```

-10- ~~*Requires:*~~*Preconditions:* `!empty()`.

[...]

Change all Requires to Preconditions in 21.4.2.5 [string.view.modifiers]:

```
constexpr void remove_prefix(size_type n);
```

-1- ~~*Requires:*~~*Preconditions:* `n <= size()`.

-2- *Effects:* Equivalent to: `data_ += n; size_ -= n;`

```
constexpr void remove_suffix(size_type n);
```

-3- ~~*Requires:*~~*Preconditions:* `n <= size()`.

Change Requires to Preconditions in 21.4.2.6 [string.view.ops]:

-8- ~~*Requires:*~~*Preconditions:* `[s, s + rlen)` is a valid range.

Change Requires to Preconditions in 22.3.3.2.2 [conversions.string]:

-16- ~~*Requires:*~~*Preconditions:* For the first and second constructors, `pcvt != nullptr`.

Change Requires to Preconditions in 22.3.3.2.3 [conversions.buffer]:

-10- ~~*Requires:*~~*Preconditions:* `pcvt != nullptr`.

Change Requires to Preconditions in 22.4.1.3.2 [facet.ctype.char.members]:

-2- ~~*Requires:*~~*Preconditions:* `tbl` either 0 or an array of at least `table_size` elements.

Change all Requires to Preconditions in 22.4.1.4.2 [locale.codecvt.virtuals]:

-1- ~~*Requires:*~~*Preconditions:* `(from<=from_end && to<=to_end)` well-defined and `true`; `state` initialized, if at the beginning of a sequence, or else equal to the result of converting the preceding characters in the sequence.

[...]

-6- ~~*Requires:*~~*Preconditions:* `(to <= to_end)` well defined and `true`; `state` initialized, if at the beginning of a sequence, or else equal to the result of converting the preceding characters in the sequence.

[...]

-11- ~~*Requires:*~~*Preconditions:* `(from <= from_end)` well defined and `true`; `state` initialized, if at the beginning of a sequence, or else equal to the result of converting the preceding characters in the sequence.

Change Requires to Preconditions in 22.4.5.1.1 [locale.time.get.members]:

-7- ~~*Requires:*~~*Preconditions:* `[fmt, fmtend)` shall be a valid range.

Change Requires to Preconditions in 22.4.5.1.2 [locale.time.get.virtuals]:

-11- ~~*Requires:*~~*Preconditions:* `t` shall point to an object.

Change Requires to Preconditions in 22.4.7.1.2 [locale.messages.virtuals]:

-4- ~~*Requires:*~~*Preconditions:* `cat` shall be a catalog obtained from `open()` and not yet closed.

[...]

-6- ~~*Requires:*~~*Preconditions:* `cat` shall be a catalog obtained from `open()` and not yet closed.

I propose making the first row use Preconditions, to allow the conforming extension of allowing an allocator with a different `value_type`, and rebinding it. Is this desirable? Should the extension only be allowed in non-strict modes? (e.g -std=gnu++17 vs -std=c++17).

Change Requires to Preconditions in Table 106, Allocator-aware container requirements:

Expression	Return Type	Assertion/note/pre-/post-condition	Complexity
`allocator_type`	A	~~*Requires:*~~*Preconditions:* `allocator_type::value_type` is the same as `X::value_type`.	compile-time
...	...	...	...
`X(rv) X u(rv)`		~~*Requires:*~~*Preconditions:* move construction of `A` shall not exit via an exception.	constant

Change Requires to Preconditions in Table 108, Optional sequence container operations:

Expression	Return Type	Operational semantics	Container
`a.pop_front()`	`void`	Destroys the first element. ~~*Requires:*~~*pre:* `a.empty()` shall be `false`.	`deque`, `forward_list`, `list`
`a.pop_back()`	`void`	Destroys the last element. ~~*Requires:*~~*pre:* `a.empty()` shall be `false`.	`basic_string`, `deque`, `list`, `vector`

Remove redundant sentence in 23.3.7.9 [array.tuple]:

-1- *Requires:* `I < N`. ~~The program is ill-formed if I is out of bounds.%-}~~

[...]

-3- *Requires:* `I < N`. ~~The program is ill-formed if I is out of bounds.%-}~~

Change Requires to Preconditions in 23.3.9.5 [forwardlist.modifiers]:

-5- ~~*Requires:*~~*Preconditions:* `position` is `before_begin()` or is a dereferenceable iterator in the range `[begin(), end())`.

[...]

-8- ~~*Requires:*~~*Preconditions:* `position` is `before_begin()` or is a dereferenceable iterator in the range `[begin(), end())`.

[...]

-11- ~~*Requires:*~~*Preconditions:* `position` is `before_begin()` or is a dereferenceable iterator in the range `[begin(), end())`. `first` and `last` are not iterators in `*this`.

[...]

-16- ~~*Requires:*~~*Preconditions:* `position` is `before_begin()` or is a dereferenceable iterator in the range `[begin(), end())`.

[...]

-19- ~~*Requires:*~~*Preconditions:* The iterator following `position` is dereferenceable.

[...]

-23- ~~*Requires:*~~*Preconditions:* All iterators in the range `(position, last)` are dereferenceable.

Change Requires to Preconditions in 23.3.9.6 [forwardlist.ops]:

-1- ~~*Requires:*~~*Preconditions:* `position` is `before_begin()` or is a dereferenceable iterator in the range `[begin(), end())`. `get_allocator() == x.get_allocator()`. `&x != this`.

[...]

-5- ~~*Requires:*~~*Preconditions:* `position` is `before_begin()` or is a dereferenceable iterator in the range `[begin(), end())`. The iterator following `i` is a dereferenceable iterator in `x`. `get_allocator() == x.get_allocator()`.

[...]

-9- ~~*Requires:*~~*Preconditions:* `position` is `before_begin()` or is a dereferenceable iterator in the range `[begin(), end())`. `(first, last)` is a valid range in `x`, and all iterators in the range `(first, last)` are dereferenceable. `position` is not an iterator in the range `(first, last)`. `get_allocator() == x.get_allocator()`.

[...]

-19- ~~*Requires:*~~*Preconditions:* `comp` defines a strict weak ordering (25.5), and `*this` and `x` are both sorted according to this ordering. `get_allocator() == x.get_allocator()`.

[...]

-23- ~~*Requires:*~~*Preconditions:* `operator<` (for the version with no arguments) or `comp` (for the version with a comparison argument) defines a strict weak ordering (25.5).

Change Requires to Preconditions in 23.3.10.5 [list.ops]:

-3- ~~*Requires:*~~*Preconditions:* `&x != this`.

[...]

-8- ~~*Requires:*~~*Preconditions:* `i` is a valid dereferenceable iterator of `x`.

[...]

-12- ~~*Requires:*~~*Preconditions:* `[first, last)` is a valid range in `x`. The result is undefined if `position` is an iterator in the range `[first, last)`. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements will continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

[...]

-22- ~~*Requires:*~~*Preconditions:* `comp` shall define a strict weak ordering (25.5), and both the list and the argument list shall be sorted according to this ordering.

[...]

-28- ~~*Requires:*~~*Preconditions:* `operator<` (for the first version) or `comp` (for the second version) shall define a strict weak ordering (25.5).

Change Requires to Preconditions in 23.6.5.1 [priqueue.cons]:

-1- ~~*Requires:*~~*Preconditions:* `x` shall define a strict weak ordering (25.5).

[...]

-3- ~~*Requires:*~~*Preconditions:* `x` shall define a strict weak ordering (25.5).

Change Requires to Preconditions in 24.4.4 [iterator.operations]:

-2- ~~*Requires:*~~*Preconditions:* `n` shall be negative only for bidirectional and random access iterators.

[...]

-5- ~~*Requires:*~~*Preconditions:* If `InputIterator` meets the requirements of random access iterator, `last` shall be reachable from `first` or `first` shall be reachable from `last`; otherwise, `last` shall be reachable from `first`.

Change Requires to Preconditions in 24.6.1.2 [istream.iterator.ops]:

-3- ~~*Requires:*~~*Preconditions:* `in_stream != 0`.

[...]

-6- ~~*Requires:*~~*Preconditions:* `in_stream != 0`.

Change Requires to Preconditions in 24.6.4.1 [ostreambuf.iter.cons]:

-1- ~~*Requires:*~~*Preconditions:* `s.rdbuf()` shall not be a null pointer.

[...]

-3- ~~*Requires:*~~*Preconditions:* `s` shall not be a null pointer.

Change Requires to Preconditions in 25.3.4 [alg.foreach]:

-12- ~~*Requires:*~~*Preconditions:* `n >= 0`.

[...]

-17- ~~*Requires:*~~*Preconditions:* `n >= 0`.

Split Requires in 25.3.12 [alg.is_permutation]:

-1- *Requires:* `ForwardIterator1` and `ForwardIterator2` shall have the same value type. ~~The comparison function shall be an equivalence relation.~~

-?- *Preconditions:* The comparison function shall be an equivalence relation.

Change Requires to Preconditions in 25.4.1 [alg.copy]:

-3- ~~*Requires:*~~*Preconditions:* `result` shall not be in the range `[first, last)`.

[...]

-8- ~~*Requires:*~~*Preconditions:* The ranges `[first, last)` and `[result, result + (last - first))` shall not overlap.

[...]

-14- ~~*Requires:*~~*Preconditions:* `result` shall not be in the range `(first, last]`.

Change Requires to Preconditions in 25.4.2 [alg.move]:

-3- ~~*Requires:*~~*Preconditions:* `result` shall not be in the range `[first, last)`.

[...]

-6- ~~*Requires:*~~*Preconditions:* The ranges `[first, last)` and `[result, result + (last - first))` shall not overlap.

Change Requires to Preconditions in 25.4.3 [alg.swap]:

-2- ~~*Requires:*~~*Preconditions:* The two ranges `[first1, last1)` and `[first2, first2 + (last1 - first1))` shall not overlap. `*(first1 + n)` shall be swappable with (17.6.3.2) `*(first2 + n)`.

[...]

-6- ~~*Requires:*~~*Preconditions:* `a` and `b` shall be dereferenceable. `*a` shall be swappable with (17.6.3.2) `*b`.

Change Requires to Preconditions in 25.4.4 [alg.transform]:

-2- ~~*Requires:*~~*Preconditions:* `op` and `binary_op` shall not invalidate iterators or subranges, or modify elements in the ranges

[...]

Split Requires in 25.4.5 [alg.replace]:

-4- Requires: The results of the expressions `*first` and `new_value` shall be writable (24.2.1) to the `result` output iterator. ~~The ranges `[first, last)` and `[result, result + (last - first))` shall not overlap.~~

-?- *Preconditions:* The ranges `[first, last)` and `[result, result + (last - first))` shall not overlap.

Split Requires in 25.4.8 [alg.remove]:

-7- Requires: ~~The ranges `[first, last)` and `[result, result + (last - first))` shall not overlap.~~ The expression `*result = *first` shall be valid.

-?- *Preconditions:* The ranges `[first, last)` and `[result, result + (last - first))` shall not overlap.

Split Requires in 25.4.9 [alg.unique]:

-2- Requires: ~~The comparison function shall be an equivalence relation.~~ The type of `*first` shall satisfy the `MoveAssignable` requirements (Table 22).

-?- *Preconditions:* The comparison function shall be an equivalence relation.

[...]

-5- Requires: ~~The comparison function shall be an equivalence relation. The ranges `[first, last)` and `[result, result+(last-first))` shall not overlap.~~ The expression `*result = *first` shall be valid. Let `T` be the value type of `InputIterator`. If `InputIterator` meets the forward iterator requirements, then there are no additional requirements for `T`. Otherwise, if `OutputIterator` meets the forward iterator requirements and its value type is the same as `T`, then `T` shall be `CopyAssignable` (Table 23). Otherwise, `T` shall be both `CopyConstructible` (Table 21) and `CopyAssignable`.

-?- *Preconditions:* The comparison function shall be an equivalence relation. The ranges `[first, last)` and `[result, result+(last-first))` shall not overlap.

Change Requires to Preconditions in 25.4.10 [alg.reverse]:

-2- ~~*Requires:*~~*Preconditions:* `*first` shall be swappable (17.6.3.2).

[...]

-5- ~~*Requires:*~~*Preconditions:* The ranges `[first, last)` and `[result, result+(last-first))` shall not overlap.

Split Requires in 25.4.11 [alg.rotate]:

-4- *Requires:* ~~`[first, middle)` and `[middle, last)` shall be valid ranges. `ForwardIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).~~ The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and the requirements of `MoveAssignable` (Table 22).

-?- *Preconditions:* `[first, middle)` and `[middle, last)` shall be valid ranges. `ForwardIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).

[...]

-8- ~~*Requires:*~~*Preconditions:* The ranges `[first, last)` and `[result, result + (last - first))` shall not overlap.

Split Requires in 25.4.12 [alg.random.sample]:

-1- *Requires:*
-- PopulationIterator shall satisfy the requirements of an input iterator (24.2.3).
-- SampleIterator shall satisfy the requirements of an output iterator (24.2.4).
-- SampleIterator shall satisfy the additional requirements of a random access iterator (24.2.7). unless PopulationIterator satisfies the additional requirements of a forward iterator (24.2.5).
-- PopulationIterator's value type shall be writable (24.2.1) to out.
-- Distance shall be an integer type.

-?- *Preconditions:*

-- UniformRandomNumberGenerator shall meet the requirements of a uniform random number generator type (26.6.1.3) whose return type is convertible to Distance.
-- out shall not be in the range [first, last).

Change Requires to Preconditions in 25.4.13 [alg.random.shuffle]:

-2- ~~*Requires:*~~*Preconditions:* `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2). The type `UniformRandomNumberGenerator` shall meet the requirements of a uniform random number generator (26.6.1.3) type whose return type is convertible to `iterator_traits<RandomAccessIterator>::difference_type`.

Change/split Requires to Preconditions in 25.4.14 [alg.partitions]:

-6- ~~*Requires:*~~*Preconditions:* `ForwardIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).

[...]

-10- Requires: ~~`BidirectionalIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).~~ The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

-?- *Preconditions:* `BidirectionalIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).

[...]

-12- Requires: `InputIterator`'s value type shall be `CopyAssignable`, and shall be writable (24.2.1) to the `out_true` and `out_false` `OutputIterator`s, and shall be convertible to `Predicate`'s argument type. ~~The input range shall not overlap with either of the output ranges.~~

-?- *Preconditions:* The input range shall not overlap with either of the output ranges.

[...]

-16- Requires: `ForwardIterator`'s value type shall be convertible to `Predicate`'s argument type. ~~`[first, last)` shall be partitioned by `pred`, i.e. all elements that satisfy `pred` shall appear before those that do not.~~

-?- *Preconditions:* `[first, last)` shall be partitioned by `pred`, i.e. all elements that satisfy `pred` shall appear before those that do not.

Split Requires in 25.5.1.1 [sort]:

-2- *Requires:* ~~`RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).~~ The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

-?- *Preconditions:* `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).

Split Requires in 25.5.1.2 [stable.sort]:

-2- *Requires:* ~~`RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).~~ The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

-?- *Preconditions:* `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).

Split Requires in 25.5.1.3 [partial.sort]:

-2- *Requires:* ~~`RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).~~ The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

-?- *Preconditions:* `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).

Split Requires in 25.5.1.4 [partial.sort.copy]:

-3- *Requires:* ~~`RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).~~ The type of `*result_first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

-?- *Preconditions:* `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).

Split Requires in 25.5.2 [alg.nth.element]:

-2- *Requires:* ~~`RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).~~ The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

-?- *Preconditions:* `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).

Change Requires to Preconditions in 25.5.3.1 [lower.bound]:

-1- ~~*Requires:*~~*Preconditions:* The elements `e` of `[first, last)` shall be partitioned with respect to the expression `e < value` or `comp(e, value)`.

Change Requires to Preconditions in 25.5.3.2 [upper.bound]:

-1- ~~*Requires:*~~*Preconditions:* The elements `e` of `[first, last)` shall be partitioned with respect to the expression `!(value < e)` or `!comp(value, e)`.

Change Requires to Preconditions in 25.5.3.3 [equal.range]:

-1- ~~*Requires:*~~*Preconditions:* The elements `e` of `[first, last)` shall be partitioned with respect to the expressions `e < value` and `!(value < e)` or `comp(e, value)` and `!comp(value, e)`. Also, for all elements `e` of `[first, last)`, `e < value` shall imply `!(value < e)` or `comp(e, value)` shall imply `!comp(value, e)`.

Change Requires to Preconditions in 25.5.3.4 [binary.search]:

-1- ~~*Requires:*~~*Preconditions:* The elements `e` of `[first, last)` are partitioned with respect to the expressions `e < value`

and `!(value < e)` or `comp(e, value)` and `!comp(value, e)`. Also, for all elements `e` of `[first, last)`, `e < value` implies `!(value < e)` or `comp(e, value)` implies `!comp(value, e)`.

Change/split Requires to Preconditions in 25.5.4 [alg.merge]:

-2- ~~*Requires:*~~*Preconditions:* The ranges `[first1, last1)` and `[first2, last2)` shall be sorted with respect to `operator<` or `comp`. The resulting range shall not overlap with either of the original ranges.

[...]

-7- ~~*Requires:*~~*Preconditions:* ~~The ranges `[first, middle)` and `[middle, last)` shall be sorted with respect to `operator<` or `comp`.~~ `BidirectionalIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2). The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

-?- *Preconditions:* The ranges `[first, middle)` and `[middle, last)` shall be sorted with respect to `operator<` or `comp`.

Change Requires to Preconditions in 25.5.5.2 [set.union]:

-2- ~~*Requires:*~~*Preconditions:* The resulting range shall not overlap with either of the original ranges.

Change Requires to Preconditions in 25.5.5.3 [set.intersection]:

-2- ~~*Requires:*~~*Preconditions:* The resulting range shall not overlap with either of the original ranges.

Change Requires to Preconditions in 25.5.5.4 [set.difference]:

-2- ~~*Requires:*~~*Preconditions:* The resulting range shall not overlap with either of the original ranges.

Change Requires to Preconditions in 25.5.5.5 [set.symmetric.difference]:

-2- ~~*Requires:*~~*Preconditions:* The resulting range shall not overlap with either of the original ranges.

Split Requires in 25.5.6.1 [push.heap]:

-2- *Requires:* ~~The range `[first, last - 1)` shall be a valid heap.~~ The type of `*first` shall satisfy the `MoveConstructible` requirements (Table 20) and the `MoveAssignable` requirements (Table 22).

-?- *Preconditions:* The range `[first, last - 1)` shall be a valid heap.

Split Requires in 25.5.6.2 [pop.heap]:

-1- *Requires:* ~~The range `[first, last)` shall be a valid non-empty heap. `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).~~ The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

-?- *Preconditions:* The range `[first, last)` shall be a valid non-empty heap. `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).

Split Requires in 25.5.6.4 [sort.heap]:

-2- *Requires:* ~~The range `[first, last)` shall be a valid heap. `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).~~ The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

-?- *Preconditions:* The range `[first, last)` shall be a valid heap. `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).

Split Requires in 25.5.7 [alg.min.max]:

-5- *Requires:* `T` shall be `CopyConstructible` ~~and `t.size() > 0`~~. For the first form, type `T` shall be `LessThanComparable`.

-?- *Preconditions:* `t.size() > 0`

[...]

-13- *Requires:* `T` shall be `CopyConstructible` ~~and `t.size() > 0`~~. For the first form, type `T` shall be `LessThanComparable`.

-?- *Preconditions:* `t.size() > 0`

[...]

-21- *Requires:* `T` shall be `CopyConstructible` ~~and `t.size() > 0`~~. For the first form, type `T` shall be `LessThanComparable`.

-?- *Preconditions:* `t.size() > 0`

Change Requires to Preconditions in 25.5.10 [alg.permutation.generators]:

-2- ~~*Requires:*~~*Preconditions:* `BidirectionalIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).

[...]

-6- ~~*Requires:*~~*Preconditions:* `BidirectionalIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).

Change Requires to Preconditions in 26.5.7 [complex.value.ops]:

-9- ~~*Requires:*~~*Preconditions:* `rho` shall be non-negative and non-NaN. `theta` shall be finite.

Change Requires to Preconditions in 26.6.8.2.1 [rand.dist.uni.int]:

-2- ~~*Requires:*~~*Preconditions:* `a ≤ b`.

Change Requires to Preconditions in 26.6.8.2.2 [rand.dist.uni.real]:

-2- ~~*Requires:*~~*Preconditions:* `a ≤ b` and `b - a ≤ numeric_limits<RealType>::max()`.

Change Requires to Preconditions in 26.6.8.3.1 [rand.dist.bern.bernoulli]:

-2- ~~*Requires:*~~*Preconditions:* `0 ≤ p ≤ 1`.

Change Requires to Preconditions in 26.6.8.3.2 [rand.dist.bern.bin]:

-2- ~~*Requires:*~~*Preconditions:* `0 ≤ p ≤ 1` and `0 ≤ t`.

Change Requires to Preconditions in 26.6.8.3.3 [rand.dist.bern.geo]:

-2- ~~*Requires:*~~*Preconditions:* `0 < p < 1`.

Change Requires to Preconditions in 26.6.8.3.4 [rand.dist.bern.negbin]:

-2- ~~*Requires:*~~*Preconditions:* `0 < p ≤ 1` and `0 < k`.

Change Requires to Preconditions in 26.6.8.4.1 [rand.dist.pois.poisson]:

-2- ~~*Requires:*~~*Preconditions:* `0 < mean`.

Change Requires to Preconditions in 26.6.8.4.2 [rand.dist.pois.exp]:

-2- ~~*Requires:*~~*Preconditions:* `0 < lambda`.

Change Requires to Preconditions in 26.6.8.4.3 [rand.dist.pois.gamma]:

-2- ~~*Requires:*~~*Preconditions:* `0 < alpha` and `0 < beta`.

Change Requires to Preconditions in 26.6.8.4.4 [rand.dist.pois.weibull]:

-2- ~~*Requires:*~~*Preconditions:* `0 < a` and `0 < b`.

Change Requires to Preconditions in 26.6.8.4.5 [rand.dist.pois.extreme]:

-2- ~~*Requires:*~~*Preconditions:* `0 < b`.

Change Requires to Preconditions in 26.6.8.5.1 [rand.dist.norm.normal]:

-2- ~~*Requires:*~~*Preconditions:* `0 < stddev`.

Change Requires to Preconditions in 26.6.8.5.2 [rand.dist.norm.lognormal]:

-2- ~~*Requires:*~~*Preconditions:* `0 < s`.

Change Requires to Preconditions in 26.6.8.5.3 [rand.dist.norm.chisq]:

-2- ~~*Requires:*~~*Preconditions:* `0 < n`.

Change Requires to Preconditions in 26.6.8.5.4 [rand.dist.norm.cauchy]:

-2- ~~*Requires:*~~*Preconditions:* `0 < b`.

Change Requires to Preconditions in 26.6.8.5.5 [rand.dist.norm.fisher]:

-2- ~~*Requires:*~~*Preconditions:* `0 < m` and `0 < n`.

Change Requires to Preconditions in 26.6.8.5.6 [rand.dist.norm.t]:

-2- ~~*Requires:*~~*Preconditions:* `0 < n`.

Split Requires in 26.6.8.6.1 [rand.dist.samp.discrete]:

-4- *Requires:* `InputIterator` shall satisfy the requirements of an input iterator (Table 114) type. Moreover, `iterator_traits<InputIterator>::value_type` shall denote a type that is convertible to `double`. ~~If `firstW == lastW`, let *n* = 1 and *w0* = 1. Otherwise, `[firstW, lastW)` shall form a sequence *w* of length *n* > 0.~~

-?- *Preconditions:* If `firstW == lastW`, let *n* = 1 and *w0* = 1. Otherwise, `[firstW, lastW)` shall form a sequence *w* of length *n* > 0.

[...]

-7- *Requires:* Each instance of type `UnaryOperation` shall be a function object (20.12) whose return type shall be convertible to `double`. Moreover, `double` shall be convertible to the type of `UnaryOperation`'s sole parameter. ~~If `nw` = 0, let *n* = 1, otherwise let *n* = `nw`. The relation 0 < δ = (`xmax` − `xmin`)/*n* shall hold.~~

-?- *Preconditions:* If `nw` = 0, let *n* = 1, otherwise let *n* = `nw`. The relation 0 < δ = (`xmax` − `xmin`)/*n* shall hold.

Split Requires in 26.6.8.6.2 [rand.dist.samp.pconst]:

-4- *Requires:* `InputIteratorB` and `InputIteratorW` shall each satisfy the requirements of an input iterator (Table 114) type. Moreover, `iterator_traits<InputIteratorB>::value_type` and `iterator_traits<InputIteratorW>::value_type` shall each denote a type that is convertible to `double`. ~~If `firstB == lastB` or `++firstB == lastB`, let *n* = 1, *w0* = 1, *b0* = 0, and *b1* = 1. Otherwise, `[firstB, lastB)` shall form a sequence *b* of length *n* + 1, the length of the sequence *w* starting from `firstW` shall be at least *n*, and any *wk* for *k* ≥ *n* shall be ignored by the distribution.~~

-?- *Preconditions:* If `firstB == lastB` or `++firstB == lastB`, let *n* = 1, *w0* = 1, *b0* = 0, and *b1* = 1. Otherwise, `[firstB, lastB)` shall form a sequence *b* of length *n* + 1, the length of the sequence *w* starting from `firstW` shall be at least *n*, and any *wk* for *k* ≥ *n* shall be ignored by the distribution.

[...]

-9- *Requires:* Each instance of type `UnaryOperation` shall be a function object (20.12) whose return type shall be convertible to `double`. Moreover, `double` shall be convertible to the type of `UnaryOperation`'s sole parameter. ~~If `nw` = 0, let *n* = 1, otherwise let *n* = `nw`. The relation 0 < δ = (`xmax` − `xmin`)/*n* shall hold.~~

-?- *Preconditions:* If `nw` = 0, let *n* = 1, otherwise let *n* = `nw`. The relation 0 < δ = (`xmax` − `xmin`)/*n* shall hold.

Split Requires in 26.6.8.6.3 [rand.dist.samp.plinear]:

-4- *Requires:* `InputIteratorB` and `InputIteratorW` shall each satisfy the requirements of an input iterator (Table 114) type. Moreover, `iterator_traits<InputIteratorB>::value_type` and `iterator_traits<InputIteratorW>::value_type` shall each denote a type that is convertible to `double`. ~~If `firstB == lastB` or `++firstB == lastB`, let *n* = 1, ρ0 = ρ1 = 1, *b0* = 0, and *b1* = 1. Otherwise, [firstB, lastB) shall form a sequence *b* of length *n* + 1, the length of the sequence *w* starting from `firstW` shall be at least *n* + 1, and any *wk* for *k* ≥ *n* + 1 shall be ignored by the distribution.~~

-?- *Preconditions:* If `firstB == lastB` or `++firstB == lastB`, let *n* = 1, ρ0 = ρ1 = 1, *b0* = 0, and *b1* = 1. Otherwise, [firstB, lastB) shall form a sequence *b* of length *n* + 1, the length of the sequence *w* starting from `firstW` shall be at least *n* + 1, and any *wk* for *k* ≥ *n* + 1 shall be ignored by the distribution.

[...]

-9- *Requires:* Each instance of type `UnaryOperation` shall be a function object (20.12) whose return type shall be convertible to `double`. Moreover, `double` shall be convertible to the type of `UnaryOperation`'s sole parameter. ~~If `nw` = 0, let *n* = 1, otherwise let *n* = `nw`. The relation 0 < δ = (`xmax` − `xmin`)/*n* shall hold.~~

-?- *Preconditions:* If `nw` = 0, let *n* = 1, otherwise let *n* = `nw`. The relation 0 < δ = (`xmax` − `xmin`)/*n* shall hold.

Change Requires to Preconditions in 26.7.2.3 [valarray.assign]:

-8- ~~*Requires:*~~*Preconditions:* The length of the array to which the argument refers equals `size()`.

Split Requires in 26.8.2 [accumulate]:

-2- *Requires:* `T` shall meet the requirements of `CopyConstructible` (Table 21) and `CopyAssignable` (Table 23) types. ~~In the range `[first, last]`, `binary_op` shall neither modify elements nor invalidate iterators or subranges.~~

-?- *Preconditions:* In the range `[first, last]`, `binary_op` shall neither modify elements nor invalidate iterators or subranges.

Change Requires to Preconditions in 26.8.3 [reduce]:

-4- ~~*Requires:*~~*Preconditions:* In the range `[first, last]`, `binary_op` shall neither modify elements nor invalidate iterators or subranges.

Change Requires to Preconditions in 26.8.4 [transform.reduce]:

-2- ~~*Requires:*~~*Preconditions:* Neither `unary_op` nor `binary_op` shall invalidate subranges, or modify elements in the range `[first, last)`.

Split Requires in 26.8.5 [inner.product]:

-2- *Requires:* `T` shall meet the requirements of `CopyConstructible` (Table 21) and `CopyAssignable` (Table 23) types. ~~In the ranges `[first1, last1]` and `[first2, first2 + (last1 - first1)]` `binary_op1` and `binary_op2` shall neither modify elements nor invalidate iterators or subranges.~~

-?- *Preconditions:* In the ranges `[first1, last1]` and `[first2, first2 + (last1 - first1)]` `binary_op1` and `binary_op2` shall neither modify elements nor invalidate iterators or subranges.

Split Requires in 26.8.6 [partial.sum]:

-4- *Requires:* `InputIterator`'s value type shall be constructible from the type of `*first`. The result of the expression `acc + *i` or `binary_op(acc, *i)` shall be implicitly convertible to `InputIterator`'s value type. `acc` shall be writable (24.2.1) to the result output iterator. ~~In the ranges `[first, last]` and `[result, result + (last - first)]` `binary_op` shall neither modify elements nor invalidate iterators or subranges.~~

-?- *Preconditions:* In the ranges `[first, last]` and `[result, result + (last - first)]` `binary_op` shall neither modify elements nor invalidate iterators or subranges.

Change Requires to Preconditions in 26.8.7 [exclusive.scan]:

-4- ~~*Requires:*~~*Preconditions:* `binary_op` shall neither invalidate iterators or subranges, nor modify elements in the ranges `[first, last)` or `[result, result + (last - first))`.

Change Requires to Preconditions in 26.8.8 [inclusive.scan]:

-4- ~~*Requires:*~~*Preconditions:* `binary_op` shall not invalidate iterators or subranges, nor modify elements in the ranges `[first, last)` or `[result, result + (last - first))`.

Change Requires to Preconditions in 26.8.9 [transform.exclusive.scan]:

-3- ~~*Requires:*~~*Preconditions:* Neither `unary_op` nor `binary_op` shall invalidate iterators or subranges, or modify elements in the ranges `[first, last)` or `[result, result + (last - first))`.

Change Requires to Preconditions in 26.8.10 [transform.inclusive.scan]:

-3- ~~*Requires:*~~*Preconditions:* Neither `unary_op` nor `binary_op` shall invalidate iterators or subranges, or modify elements in the ranges `[first, last)` or `[result, result + (last - first))`.

Split Requires in 26.8.11 [adjacent.difference]:

-2- *Requires:* `InputIterator`'s value type shall be `MoveAssignable` (Table 22) and shall be constructible from the type of `*first`. `acc` shall be writable (24.2.1) to the result output iterator. The result of the expression `val - acc` or `binary_op(val, acc)` shall be writable to the result output iterator. ~~In the ranges [first, last] and [result, result + (last - first)], binary_op shall neither modify elements nor invalidate iterators or subranges.~~

-?- *Preconditions:* In the ranges [first, last] and [result, result + (last - first)], binary_op shall neither modify elements

nor invalidate iterators or subranges.

Change Requires to Preconditions in 27.5.3.6 [ios.base.callbacks]:

-2- ~~*Requires:*~~*Preconditions:* The function `fn` shall not throw exceptions.

Change Requires to Preconditions in 27.5.5.3 [basic.ios.members]:

-2- ~~*Requires:*~~*Preconditions:* If `tiestr` is not null, `tiestr` must not be reachable by traversing the linked list of tied stream objects starting from `tiestr->tie()`.

[...]

-22- ~~*Requires:*~~*Preconditions:* `sb != nullptr`.

Change Requires to Preconditions in 27.6.3.4.3 [streambuf.virt.get]:

-15- ~~*Requires:*~~*Preconditions:* The constraints are the same as for `underflow()`, except that the result character shall be transferred from the pending sequence to the backup sequence, and the pending sequence shall not be empty before the transfer.

Change Requires to Preconditions in 27.6.3.4.5 [streambuf.virt.put]:

-5- ~~*Requires:*~~*Preconditions:* Every overriding definition of this virtual function shall obey the following constraints:

Change Requires to Preconditions in 27.6.3.4.5 [streambuf.virt.put]:

-3- ~~*Requires:*~~*Preconditions:* `s` shall not be a null pointer.

Change Requires to Preconditions in 27.7.5 [ext.manip]:

-7- ~~*Requires:*~~*Preconditions:* The argument `tmb` shall be a valid pointer to an object of type `struct tm`, and the argument `fmt` shall be a valid pointer to an array of objects of type `charT` with `char_traits<charT>::length(fmt)` elements.

[...]

-9- ~~*Requires:*~~*Preconditions:* The argument `tmb` shall be a valid pointer to an object of type `struct tm`, and the argument `fmt` shall be a valid pointer to an array of objects of type `charT` with `char_traits<charT>::length(fmt)` elements.

Change Requires to Preconditions in 27.9.2.4 [filebuf.virtuals]:

-20- ~~*Requires:*~~*Preconditions:* If the file is not positioned at its beginning and the encoding of the current locale as determined by `a_codecvt.encoding()` is state-dependent (22.4.1.4.2) then that facet is the same as the corresponding facet of `loc`.

Change Requires to Preconditions in 27.10.2.3 [fs.race.behavior]:

-2- If the possibility of a file system race would make it unreliable for a program to test for a precondition before calling a function described herein, ~~*Requires:*~~*Preconditions:* is not specified for the function. [*Note:* As a design practice, preconditions are not specified when it is unreasonable for a program to detect them prior to calling the function. *--end note*]

Split Requires in 27.10.8.6.2 [path.factory]:

-1- *Requires:* ~~The `source` and `[first, last)` sequences are UTF-8 encoded.~~ The value type of `Source` and `InputIterator` is `char`.

-?- *Preconditions:* The `source` and `[first, last)` sequences are UTF-8 encoded.

The preconditions on the following `recursive_directory_iterator` member functions changed in Oulu, but should still become *Preconditions:* instead of *Requires:*

Change Requires to Preconditions in 27.10.14.1 [rec.dir.itr.members]:

-17- ~~*Requires:*~~*Preconditions:* `*this != recursive_directory_iterator()`.

[...]

-20- ~~*Requires:*~~*Preconditions:* `*this != recursive_directory_iterator()`.

[...]

-23- ~~*Requires:*~~*Preconditions:* `*this != recursive_directory_iterator()`.

[...]

-26- ~~*Requires:*~~*Preconditions:* `*this != recursive_directory_iterator()`.

[...]

-30- ~~*Requires:*~~*Preconditions:* `*this != recursive_directory_iterator()`.

[...]

-32- ~~*Requires:*~~*Preconditions:* `*this != recursive_directory_iterator()`.

Change Requires to Preconditions in 27.10.15.3 [fs.op.copy]:

-2- ~~*Requires:*~~*Preconditions:* At most one constant from each option group (27.10.10.2) is present in `options`.

Change Requires to Preconditions in 27.10.15.4 [fs.op.copy_file]:

-3- ~~*Requires:*~~*Preconditions:* At most one constant from each `copy_options` option group (27.10.10.2) is present in `options`.

Change Requires to Preconditions in 27.10.15.26 [fs.op.permission]:

-1- ~~*Requires:*~~*Preconditions:* `!((prms & perms::add_perms) != perms::none && (prms & perms::remove_perms) != perms::none)`.

Change Requires to Preconditions in 28.7 [re.traits]:

-13- ~~*Requires:*~~*Preconditions:* The value of *radix* shall be 8, 10, or 16.

Change Requires to Preconditions in 28.8.2 [re.regex.construct]:

-2- ~~*Requires:*~~*Preconditions:* *p* shall not be a null pointer.

[...]

-6- ~~*Requires:*~~*Preconditions:* *p* shall not be a null pointer.

Change Requires to Preconditions in 28.8.3 [re.regex.assign]:

-5- ~~*Requires:*~~*Preconditions:* `ptr` shall not be a null pointer.

Change Requires to Preconditions in 28.10.4 [re.results.acc]:

-1- ~~*Requires:*~~*Preconditions:* `ready() == true`.

[...]

-3- ~~*Requires:*~~*Preconditions:* `ready() == true`.

[...]

-5- ~~*Requires:*~~*Preconditions:* `ready() == true`.

[...]

-7- ~~*Requires:*~~*Preconditions:* `ready() == true`.

[...]

-9- ~~*Requires:*~~*Preconditions:* `ready() == true`.

[...]

-11- ~~*Requires:*~~*Preconditions:* `ready() == true`.

Change/split Requires in 28.10.5 [re.results.form]:

-1- *Requires:* ~~`ready() == true` and~~ `OutputIter` shall satisfy the requirements for an Output Iterator (24.2.4).

-?- *Preconditions:* `ready() == true`.

[...]

-3- ~~*Requires:*~~*Preconditions:* `ready() == true`.

[...]

-8- ~~*Requires:*~~*Preconditions:* `ready() == true`.

Change Requires to Preconditions in 28.11.3 [re.alg.search]:

-2- ~~*Requires:*~~*Preconditions:* Each of the initialization values of `submatches` shall be `>= -1`.

Change Requires to Preconditions in 29.6.5 [atomics.types.operations.req]:

-10- ~~*Requires:*~~*Preconditions:* The order argument shall not be `memory_order_consume`, `memory_order_acquire`, nor `memory_order_acq_rel`.

[...]

-14- ~~*Requires:*~~*Preconditions:* The order argument shall not be `memory_order_release` nor `memory_order_acq_rel`.

[...]

-21- ~~*Requires:*~~*Preconditions:* The failure argument shall not be `memory_order_release` nor `memory_order_acq_rel`. The `failure` argument shall be no stronger than the `success` argument.

Change Requires to Preconditions in 29.7 [atomics.flag]:

-7- ~~*Requires:*~~*Preconditions:* The order argument shall not be `memory_order_consume`, `memory_order_acquire`, nor `memory_order_acq_rel`.

Change Requires to Preconditions in 30.2.5.2 [thread.req.lockable.basic]:

-3- ~~*Requires:*~~*Preconditions:* The current execution agent shall hold a lock on `m`.

Change Requires to Preconditions in 30.4.1.2 [thread.mutex.requirements.mutex]:

-7- ~~*Requires:*~~*Preconditions:* If `m` is of type `std::mutex`, `std::timed_mutex`, `std::shared_mutex`, or `std::shared_timed_mutex`, the calling thread does not own the mutex.

[...]

-15- ~~*Requires:*~~*Preconditions:* If `m` is of type `std::mutex`, `std::timed_mutex`, `std::shared_mutex`, or `std::shared_timed_mutex`, the calling thread does not own the mutex.

[...]

-22- ~~*Requires:*~~*Preconditions:* The calling thread shall own the mutex.

Change Requires to Preconditions in 30.4.1.3 [thread.timedmutex.requirements]:

-4- ~~*Requires:*~~*Preconditions:* If `m` is of type `std::timed_mutex` or `std::shared_timed_mutex`, the calling thread does not own the mutex.

[...]

-11- ~~*Requires:*~~*Preconditions:* If `m` is of type `std::timed_mutex` or `std::shared_timed_mutex`, the calling thread does not own the mutex. Change Requires to Preconditions in 30.4.1.3 [thread.timedmutex.requirements]:

Change Requires to Preconditions in 30.4.1.4 [thread.sharedmutex.requirements]:

-4- ~~*Requires:*~~*Preconditions:* The calling thread has no ownership of the mutex.

[...]

-12- ~~*Requires:*~~*Preconditions:* The calling thread shall hold a shared lock on the mutex.

[...]

-17- ~~*Requires:*~~*Preconditions:* The calling thread has no ownership of the mutex.

Change Requires to Preconditions in 30.4.1.5 [thread.sharedtimedmutex.requirements]:

-3- ~~*Requires:*~~*Preconditions:* The calling thread has no ownership of the mutex.

[...]

-10- ~~*Requires:*~~*Preconditions:* The calling thread has no ownership of the mutex.

Change Requires to Preconditions in 30.4.2.1 [thread.lock.guard]:

-2- ~~*Requires:*~~*Preconditions:* If a `MutexTypes` type is not a recursive mutex, the calling thread does not own the corresponding mutex element of `m`.

[...]

-4- ~~*Requires:*~~*Preconditions:* The calling thread owns all the mutexes in `m`.

Change Requires to Preconditions in 30.4.2.2.1 [thread.lock.unique.cons]:

-3- ~~*Requires:*~~*Preconditions:* If `mutex_type` is not a recursive mutex the calling thread does not own the mutex.

[...]

-8- ~~*Requires:*~~*Preconditions:* The supplied `Mutex` type shall meet the `Lockable` requirements (30.2.5.3). If `mutex_type` is not a recursive mutex the calling thread does not own the mutex.

[...]

-11- ~~*Requires:*~~*Preconditions:* The calling thread owns the mutex.

[...]

-15- ~~*Requires:*~~*Preconditions:* If `mutex_type` is not a recursive mutex the calling thread does not own the mutex. The supplied `Mutex` type shall meet the `TimedLockable` requirements (30.2.5.4).

[...]

-18- ~~*Requires:*~~*Preconditions:* If `mutex_type` is not a recursive mutex the calling thread does not own the mutex. The supplied `Mutex` type shall meet the `TimedLockable` requirements (30.2.5.4).

Change Requires to Preconditions in 30.4.2.2.2 [thread.lock.unique.locking]:

-4- ~~*Requires:*~~*Preconditions:* The supplied `Mutex` type shall meet the `Lockable` requirements (30.2.5.3).

[...]

-9- ~~*Requires:*~~*Preconditions:* The supplied `Mutex` type shall meet the `TimedLockable` requirements (30.2.5.4).

[...]

-14- ~~*Requires:*~~*Preconditions:* The supplied `Mutex` type shall meet the `TimedLockable` requirements (30.2.5.4).

Change Requires to Preconditions in 30.4.2.3.1 [thread.lock.shared.cons]:

-3- ~~*Requires:*~~*Preconditions:* The calling thread does not own the mutex for any ownership mode.

[...]

-8- ~~*Requires:*~~*Preconditions:* The calling thread does not own the mutex for any ownership mode.

[...]

-11- ~~*Requires:*~~*Preconditions:* The calling thread has shared ownership of the mutex.

[...]

-14- ~~*Requires:*~~*Preconditions:* The calling thread does not own the mutex for any ownership mode.

[...]

-17- ~~*Requires:*~~*Preconditions:* The calling thread does not own the mutex for any ownership mode.

Change Requires to Preconditions in 30.4.3 [thread.lock.algorithm]:

-1- ~~*Requires:*~~*Preconditions:* Each template parameter type shall meet the `Lockable` requirements. [*Note:* The `unique_lock` class template meets these requirements when suitably instantiated. *--end note*]

[...]

-4- ~~*Requires:*~~*Preconditions:* Each template parameter type shall meet the `Lockable` requirements. [*Note:* The `unique_lock` class template meets these requirements when suitably instantiated. *--end note*]

Change Requires to Preconditions in 30.5 [thread.condition]:

-6- ~~*Requires:*~~*Preconditions:* `lk` is locked by the calling thread and either
— no other thread is waiting on `cond`, or
— `lk.mutex()` returns the same value for each of the lock arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

Change Requires to Preconditions in 30.5.1 [thread.condition.condvar]:

-5- ~~*Requires:*~~*Preconditions:* There shall be no thread blocked on `*this`. [*Note:* That is, all threads shall have been notified; they may subsequently block on the lock specified in the wait. This relaxes the usual rules, which would have required all wait calls to happen before destruction. Only the notification to unblock the wait must happen before destruction. The user must take care to ensure that no threads wait on `*this` once the destructor has been started, especially when the waiting threads are calling the wait functions in a loop or using the overloads of `wait`, `wait_for`, or `wait_until` that take a predicate. *--end note*]

[...]

-9- ~~*Requires:*~~*Preconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either
— no other thread is waiting on this `condition_variable` object or
— `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

[...]

-14- ~~*Requires:*~~*Preconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either
— no other thread is waiting on this `condition_variable` object or
— `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

[...]

-19- ~~*Requires:*~~*Preconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either
— no other thread is waiting on this `condition_variable` object or
— `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

[...]

-25- ~~*Requires:*~~*Preconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either
— no other thread is waiting on this `condition_variable` object or
— `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

[...]

-31- ~~*Requires:*~~*Preconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either
— no other thread is waiting on this `condition_variable` object or
— `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

[...]

-37- ~~*Requires:*~~*Preconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either
— no other thread is waiting on this `condition_variable` object or
— `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`,

`wait_for`, or `wait_until`) threads.

Change Requires to Preconditions in 30.5.2 [thread.condition.condvarany]:

-5- ~~*Requires:*~~*Preconditions:* There shall be no thread blocked on `*this`. [*Note:* That is, all threads shall have been notified; they may subsequently block on the lock specified in the wait. This relaxes the usual rules, which would have required all wait calls to happen before destruction. Only the notification to unblock the wait must happen before destruction. The user must take care to ensure that no threads wait on `*this` once the destructor has been started, especially when the waiting threads are calling the wait functions in a loop or using the overloads of `wait`, `wait_for`, or `wait_until` that take a predicate. --*end note*]

Change Requires to Preconditions in 30.6.5 [futures.promise]:

-3- ~~*Requires:*~~*Preconditions:* `Alloc` shall be an Allocator (17.6.3.5).

[...]

-18- ~~*Requires:*~~*Preconditions:* `p` is not null.

[...]

-25- ~~*Requires:*~~*Preconditions:* `p` is not null.

Split Requires in 30.6.9.1 [futures.task.members]:

-2- Requires: *INVOKE*`(f, t1, t2, ..., tN, R)`, where `t1`, `t2`, `...`, `tN` are values of the corresponding types in `ArgTypes...`, shall be a valid expression. ~~Invoking a copy of `f` shall behave the same as invoking `f`.~~

-?- *Preconditions:* Invoking a copy of `f` shall behave the same as invoking `f`.

Change Requires to Preconditions in 30.6.9.2 [futures.task.nonmembers]:

-2- ~~*Requires:*~~*Preconditions:* `Alloc` shall be an Allocator (17.6.3.5).